

Previously...

Phase 2 Detailed Task Distribution

As we step into Phase 2 (Automated RLAIF Data Engine, NLI Calibration, and Edge Generation), here is our exact division of labor:

M1: Khushi (Lead Architect & Core Implementation)

- **Three-Way Decision Policy Integration:** Upgrade the cross-encoder verification gate from a binary pass/fail check into the full three-way deterministic state machine routing: **ANSWER** ($P(\text{Entailment}) \geq 0.75$), **CLARIFY** ($P(\text{Neutral}) > 0.40$), and **ABSTAIN** ($P(\text{Contradiction}) > 0.30$).
- **Empirical Ablation Framework:** Write the evaluation execution script to run systematic threshold sweeps across our benchmark queries to map precision, recall, and selective risk curves.
- **Module Integration & Orchestration:** Merge Praisys's RLAIF dataset generation script and Meghana's GBNF generation module into the central master repository.
- **Paper Drafting:** Begin writing core sections of the research manuscript (*Introduction, Methodology, and Experimental Setup*).

M2: Meghana (Development Support: SLM Generation, GBNF & UI)

- **GBNF Grammar Schema Construction:** Create the constraints.gbnf context-free grammar file to enforce strict JSON structure compliance on the Qwen-2.5-1.5B model outputs (including fields for `evidence_chunk_id`, `parameter_identified`, and `recommended_action_telugu`).
- **Local SLM Engine Integration:** Wire up llama.cpp Python bindings with the quantized Qwen-2.5-1.5B-Instruct model using logit-masking.
- **Streamlit Edge Interface:** Build the local browser UI (localhost:8501) featuring real-time diagnostic telemetry panels displaying retrieval scores, decision states, and JSON citation outputs.
- **Integration Testing:** Work with M1 to ensure end-to-end Time-to-First-Token (TTFT) stays under 3.0 seconds on CPU.

M3: Jyothish (Documentation, Logbook & Diagrams)

- **Engineering Logbook:** Maintain a detailed, dated record of all weekly decisions, hyperparameter configurations, challenges, and empirical solutions.
- **Zotero & Reference Management:** Curate and organize all literature survey references (ICAR-CIBA manuals, BGE-M3, DeBERTa, DPO, and RLAIF papers) for the bibliography.
- **Publication-Ready Diagrams:** Produce high-resolution vector diagrams (using Draw.io or LaTeX TikZ) for:
 1. The Two-Stage Cascaded Retrieval Pipeline.
 2. The LangGraph Three-Way Decision State Machine.
 3. The Automated RLAIF Data Engine Architecture.
- **Repository Documentation:** Keep the GitHub README.md and setup instructions fully up-to-date.

M4: Praisys (Review Support, Data Engine & Visualizations)

- **Automated RLAIF Data Engine Script** (`src/generate_rlaif_data.py`): Implement the script that synthesizes cross-lingual NLI training triplets (Entailment, Neutral, Contradiction) from processed English text chunks.
- **Biological Invariant Verification Guardrails:** Build the Python assertion suite

(`verify_aquaculture_invariants`) to filter out biologically impossible values (e.g., pH bounds outside [6.0, 9.5] or invalid dissolved oxygen levels) before dataset admission.

- **Result Visualizations & Graphs:** Generate high-resolution figures for the paper and presentations, including safety-coverage curves and latency breakdown charts.
- **Presentation, Demo & Viva Prep:** Coordinate slide decks, structure the demo evaluation flow, and compile anticipated viva questions and answers.

Phase 2 Execution Flow & Trigger Milestones

Step 1: Praisyy (The Starting Block)

- **When to start:** Immediately. Praisyy's work has zero upstream dependencies because it builds directly off the clean `chunks.json` we just finalized in Phase 1.
- **What they do:** Run `src/generate_rlaif_data.py` to ingest the chunks, synthesize the Telugu query pairs, and apply the biological invariant filter (`verify_aquaculture_invariants`).
- **The Hand-off Trigger:** Once Praisyy generates and pushes the verified training/evaluation dataset to the repository (e.g., `data/evaluation/rlaif_dataset.json`), **Khushi and Meghana are instantly unblocked to start Steps 2 and 3.**

Step 2: Khushi & Meghana (Parallel Core Development)

- **When to start:** The exact moment Praisyy's dataset file lands in the repository.
- **What Khushi does:** Takes the evaluation data and upgrades the cross-encoder verification gate into the full **Three-Way Decision Policy** (`Answer`, `Clarify`, `Abstain`) with probability thresholds.
- **What Meghana does:** Takes the output requirements and writes the `constraints.gbnf` grammar file, wires up `llama.cpp` with Qwen-2.5-1.5B, and builds the local Streamlit UI (`localhost:8501`).
- **The Hand-off Trigger:** Once Khushi locks down the routing logic and Meghana finishes the edge generation UI, the complete end-to-end pipeline is functional.

Step 3: Jyothish & Praisyy (Integration, Validation & Documentation)

- **When to start:** Runs concurrently as Step 2 wraps up, ramping up fully once the integrated pipeline is running.
- **What Jyothish does:** Documents hyperparameter logs, organizes Zotero citations, and produces the vector architecture diagrams (LangGraph State Machine, Cascaded Pipeline) as code components stabilize.
- **What Praisyy does:** Takes the final system outputs to run the ablation sweeps, generate safety-coverage curves, and compile the final presentation/viva materials.

Issues realised while building:

Decision_router.py:

NLI:

Upon quick test:

The screenshot shows a code editor with the file `decision_router.py` open. The code includes a `__main__` block for testing. The terminal output shows the execution of the script, including model downloads and the results of two tests: Entailment Match and Contradiction Match.

```
base_queries.json X embed.py X ingest.py X decision_router.py U X
src > decision_router.py > ...

55     return decision, metrics
56
57     --- Quick Local Test ---
58     if __name__ == "__main__":
59         router = NLIDecisionRouter()
60
61         test_query = "What is the optimal pH for Vannamei shrimp?"
62         test_context_entail = "For Litopenaeus vannamei culture, the pH range should be strictly maintained between 7.5 to 8.5."
63         test_context_contradict = "Vannamei shrimp require a highly acidic environment with a pH of 4.0."
64
65         print("\n[Test 1] Entailment Match:")
66         decision, metrics = router.evaluate_routing(test_query, test_context_entail)
67         print(f"Decision: {decision} | Scores: {metrics}")
68
69         print("\n[Test 2] Contradiction Match:")
70         decision, metrics = router.evaluate_routing(test_query, test_context_contradict)
71         print(f"Decision: {decision} | Scores: {metrics}")

Problems Output Debug Console Terminal ...
(zenv) (base) jolliojelly@Khushis-MacBook-Air AquaAssist % python
src/decision_router.py
spm.model: downloading bytes: 1.63MB, 154kB/s
spm.model: reconstructing file: 100%| 2.46MB / 2.46MB, 234kB/s
tokenizer.json: 100%| 8.66M/8.66M [00:00<00:00, 8.80MB/s]
added_tokens.json: 100%| 26.0/26.0 [00:00<00:00, 63.9kB/s]
special_tokens_map.json: 100%| 301/301 [00:00<00:00, 770kB/s]

[Test 1] Entailment Match:
Decision: CLARIFY | Scores: {'entailment': 0.0003, 'neutral': 0.9986, 'contradiction': 0.0011}

[Test 2] Contradiction Match:
Decision: ABSTAIN | Scores: {'entailment': 0.0, 'neutral': 0.0039, 'contradiction': 0.9961}
(zenv) (base) jolliojelly@Khushis-MacBook-Air AquaAssist %
```

```
# --- Quick Local Test ---
if __name__ == "__main__":
    router = NLIDecisionRouter()
```

```
test_query = "What is the optimal pH for Vannamei shrimp?"
test_context_entail = "For Litopenaeus vannamei culture, the pH range should be strictly maintained
between 7.5 to 8.5."
test_context_contradict = "Vannamei shrimp require a highly acidic environment with a pH of 4.0."
```

```
print("\n[Test 1] Entailment Match:")
decision, metrics = router.evaluate_routing(test_query, test_context_entail)
print(f"Decision: {decision} | Scores: {metrics}")

print("\n[Test 2] Contradiction Match:")
decision, metrics = router.evaluate_routing(test_query, test_context_contradict)
print(f"Decision: {decision} | Scores: {metrics}")
```

These scores reveal exactly how the DeBERTa Natural Language Inference (NLI) model interprets logical relationships, highlighting a massive success in the safety gate but exposing a formatting flaw in how queries are fed into the system.

Test 2: The Contradiction Safeguard (Success)

- **The Metric:** A 99.61% contradiction probability.
- **What It Tells Us:** The model flawlessly caught the biological trap. It mathematically recognized that the premise of requiring "pH 4.0" is fundamentally incompatible with the strict "7.5 to 8.5" bounds outlined in the context.
- **System Impact:** The routing logic properly hit the **ABSTAIN** state. In a live environment, this successfully stops the pipeline from generating a harmful, hallucinated recommendation.

Test 1: The Entailment Miss (The Formatting Flaw)

- **The Metric:** A 99.86% neutral probability and near-zero entailment (0.03%).
- **What It Tells Us:** The model completely failed to recognize that the context perfectly answers the query, improperly dumping a valid retrieval into the **CLARIFY** bucket instead of **ANSWER**.
- **The Root Cause:** Standard NLI models are explicitly trained on pairs of declarative statements (a Premise and a Hypothesis). Because the input was an explicit question ("What is the optimal pH...?"), the model's logic engine stalled. It evaluated the pair and logically concluded: "A question does not assert a fact, therefore it cannot prove, disprove, or entail the context."

The Fix for the Data Engine To achieve accurate three-way routing, incoming user questions must be flattened into declarative statements or core semantic phrases before hitting the cross-encoder.

Real users don't talk in robotic, declarative statements—they ask messy, natural questions. Pure NLI models (like DeBERTa) are mathematically trained to evaluate two *statements* (Premise vs. Hypothesis). When you feed them an interrogative question, their logic engine stalls and defaults to "Neutral" because a question cannot technically prove or disprove a fact.

Aquaculture workers describe symptoms and farm conditions—like "shrimp have white spots and water smells foul"—rather than querying explicit parameters. A rigid NLI gate will struggle with rambling diagnostic paragraphs unless you introduce an intermediate processing layer before the retrieval stage.

Pre-Retrieval Query Translation

- Instead of passing the raw, messy paragraph directly to ChromaDB and the DeBERTa cross-encoder, route it first to your local Qwen-2.5-1.5B model.
- Use a strict system prompt asking the SLM to act as a diagnostic extractor. It should compress the farmer's paragraph into a clean, declarative intent statement (e.g., "The farmer observes high mortality and surface swimming in Vannamei ponds").
- This clean extraction becomes the actual query fed into the BGE-M3 hybrid retriever, drastically improving semantic matching against the ICAR-CIBA manuals.

Triggering the CLARIFY State

- If a farmer provides a vague paragraph ("my shrimp are dying"), Qwen extracts the generic intent, and ChromaDB retrieves general mortality guidelines.
- When the NLI router compares "The farmer observes shrimp mortality" (Query) against "Mortality requires checking TAN and pH levels" (Context), it correctly evaluates to **Neutral** (insufficient data to provide a final answer).
- This perfectly triggers your **CLARIFY** state. Meghana's Streamlit UI, guided by the GBNF grammar, intercepts this and prompts the user in Telugu: *"To diagnose the mortality, please describe the water color or any recent pH readings."*

The End-to-End Diagnostic Flow

- **Input:** "Water turned dark green yesterday, and the fish aren't eating."
- **Translation:** "Suspected algal bloom and loss of appetite in pond."
- **Retrieval:** Pulls chunks on phytoplankton crashes and feeding protocols.
- **Decision:** NLI scores high Entailment.
- **Generation:** Qwen generates the treatment protocol based purely on the retrieved chunk.

This architectural pivot is exactly what elevates AquaAssist from a standard academic project to a production-ready, IEEE-grade diagnostic system. Real-world farmers don't speak in neat search queries; they describe symptoms.

By inserting an SLM extraction layer *before* retrieval, both the NLI gate and the vector database receive clean, declarative statements they can mathematically process. Here is the fully updated, loophole-proof system flow and the revised Phase 2 task allocation.

The Updated AquaAssist System Flow (End-to-End)

Stage 1: Ingestion & Indexing (Phase 1 - Completed)

ICAR-CIBA manuals are parsed into `chunks.json`, tagged with metadata (species, parameters, global IDs), and embedded into ChromaDB via BGE-M3 to support both semantic and exact numerical matching.

Stage 2: Intent Extraction (The Failsafe Buffer)

- **Input:** The farmer types or speaks a messy paragraph (e.g., "The water turned dark green yesterday, shrimp aren't eating, and I see white spots on their shells.")
- **Processing:** Qwen-2.5-1.5B (via local `llama.cpp`) runs a fast, strict zero-shot prompt to flatten the input into a declarative diagnostic hypothesis.
- **Output:** *"The pond exhibits a phytoplankton bloom, loss of appetite, and suspected White Spot Syndrome Virus (WSSV)."*

Stage 3: Hybrid Retrieval

ChromaDB uses BGE-M3 to map the clean hypothesis against the database, pulling the top-K relevant chunks containing exact treatment protocols and parameters.

Stage 4: The NLI Three-Way Gate (DeBERTa)

The router compares the extracted hypothesis against the retrieved chunks to determine safety and sufficiency.

- **ANSWER ($P(\text{Entailment}) \geq 0.75$):** The chunks contain a complete diagnostic match. Route to Stage 5.
- **CLARIFY ($P(\text{Neutral}) > 0.40$):** The chunks match the topic (e.g., mortality) but lack specifics to prescribe treatment. The UI intercepts and prompts the farmer: *"Please provide current pH and ammonia levels to confirm."*
- **ABSTAIN ($P(\text{Contradiction}) > 0.30$):** The farmer's described actions or conditions critically violate ICAR-CIBA invariants (e.g., "I dumped 50kg of raw manure into the tank"). The system immediately halts and issues a hard biological safety warning.

Stage 5: Guardrailed Generation

If routed to ANSWER, Qwen-2.5-1.5B generates the final treatment plan in the vernacular language, strictly constrained by a GBNF grammar schema to guarantee structural JSON output (Citations, Actions, Warnings) without hallucinated text.

About the Original proposal shared with our professor in the main gc..

The original proposal routed raw, colloquial farmer questions directly into the Natural Language Inference (NLI) evaluation gate without converting them into verifiable statements first.

The Original Architectural Flaw

- The system was designed to capture a "Telugu Voice / Colloquial Query" and transcribe it into a "Raw Vernacular String".
- This raw string was immediately sent to the "Hybrid Lexical-Semantic Retrieval" layer to pull candidate

chunks.

- The architecture then explicitly passed this raw query alongside the retrieved evidence straight into the "Custom Vernacular Cross-Encoder NLI Gate".
- The NLI gate was tasked with computing full cross-attention over the "(Query $\otimes$ Evidence)" to calculate probabilities for Entailment, Neutral, and Contradiction.

The Practical Loophole

- The proposal correctly identifies that rural farmers do not use standard academic prose, instead asking dialectal questions like "White dots appeared on shrimp and they are dying, what medicine to spray?".
- The loophole exists because NLI mathematical models cannot logically process an interrogative question against an evidence chunk; they require two declarative statements to evaluate truth.
- By feeding a conversational question directly into the NLI gate as proposed, the model's logic engine stalls, inevitably defaulting to a "Neutral" state because a question cannot technically confirm or contradict a factual chunk.

The Missing Component

- The original "Decoupled, Dual-Gated, and Grammar-Constrained Architecture" diagram completely lacked a query-rewriting or intent-extraction phase before retrieval.
- The SLM (Qwen-2.5-1.5B) was strictly locked to the final step of the pipeline exclusively for "GBNF-Constrained Structured Generation" within the "ANSWER" state.
- Because the generative model was only utilized at the very end of the flow, the pipeline had no mechanism to clean, compress, or flatten the farmer's messy diagnostic input into a usable declarative hypothesis before hitting the critical NLI verification step.

The updated plan solves the mathematical loophole by introducing a "Pre-Retrieval Extraction" layer that intercepts the query before it ever reaches the database or the NLI gate.

How the Updated Plan Fixes the System

- **Dual SLM Usage:** Instead of saving the Qwen-2.5-1.5B model strictly for the final step, the updated architecture calls it twice. It acts as a lightweight translator at the very beginning of the pipeline, and as the final answer generator at the end.
- **Declarative Flattening (The Core Fix):** When a farmer inputs a rambling, dialect-heavy question, the SLM immediately extracts the core intent and flattens it into a verifiable statement. An interrogative like *"My water is green and shrimp have white spots, what do I do?"* is instantly converted into the declarative hypothesis: *"The pond exhibits a phytoplankton bloom and suspected White Spot Syndrome Virus."*
- **Unblocking the NLI Gate:** Because the DeBERTa Cross-Encoder now receives a flat, declarative hypothesis rather than an interrogative question, its logic engine no longer stalls. It can mathematically evaluate the hypothesis against the retrieved ICAR-CIBA chunks to successfully trigger the **ANSWER**, **CLARIFY**, or **ABSTAIN** states.
- **Sharper BGE-M3 Retrieval:** The hybrid ChromaDB search becomes significantly more accurate. Instead of trying to vector-match conversational noise and question marks against academic textbooks, it searches using the clean diagnostic summary.

Revised System Flow and Phase 2 Task Distribution

The Updated AquaAssist System Flow

- **Stage 1: Ingestion & Indexing (Phase 1 - Done):** ICAR-CIBA manuals are parsed, tagged, and indexed into ChromaDB using BGE-M3 hybrid search.
- **Stage 2: Pre-Retrieval Extraction (The New Fix):** The farmer inputs a messy, conversational question. We run a lightning-fast Qwen SLM prompt to instantly flatten this into a clean, declarative hypothesis (e.g., *"The pond exhibits a phytoplankton bloom and suspected WSSV."*).
- **Stage 3: Hybrid Retrieval:** ChromaDB uses this clean hypothesis (instead of the messy question) to pull the top exact matches from the database.
- **Stage 4: The Three-Way NLI Gate:** The DeBERTa cross-encoder evaluates the hypothesis against the retrieved chunks mathematically:
 - **ANSWER (Entailment ≥ 0.75):** Perfect match. Route to generation.
 - **CLARIFY (Neutral > 0.40):** Missing info. Prompt the user for specific parameters.
 - **ABSTAIN (Contradiction > 0.30):** Biologically unsafe. Halt and refuse.
- **Stage 5: Guardrailed Generation:** Qwen SLM generates the final Telugu advice using GBNF grammar constraints to guarantee structured, hallucination-free JSON.

Updated Phase 2 Task Distribution

M1: Khushi (Lead Architect: NLI Routing & Extractor Logic)

- **The Extractor Prompt:** Write the strict zero-shot system prompt that forces Qwen to compress messy farmer paragraphs into flat, declarative hypotheses.
- **NLI Gate Integration:** Update `src/decision_router.py` so the cross-encoder evaluates the *extracted hypothesis* (not the raw question) to accurately trigger the Answer, Clarify, or Abstain states.
- **Empirical Ablation:** Feed our benchmark dataset through this newly updated pipeline and map the precision, recall, and safety-coverage curves for the IEEE paper.

M2: Meghana (Development: Dual-SLM Hooks, UI & GBNF)

- **Dual-SLM Integration:** Update the `llama.cpp` pipeline to handle two distinct Qwen calls: a fast, lightweight call for Stage 2 (Extraction) and the heavy final call for Stage 5 (Generation).
- **Conversational UI Loop:** Update the Streamlit frontend (`localhost:8501`) to support the **CLARIFY** state, allowing the system to ask the user follow-up questions without resetting their chat history.
- **GBNF Grammar:** Write the `constraints.gbnf` file to lock down the final generation into strict JSON (Citations, Actions, Warnings).

M3: Jyothish (Documentation, Diagrams & Hardware Logging)

- **Updated Pipeline Diagrams:** Redraw the system architecture diagrams (Draw.io/TikZ) to explicitly show the new "Pre-Retrieval Extractor" layer feeding into the ChromaDB/DeBERTa pipeline.
- **Latency Profiling:** Track the new Time-to-First-Token (TTFT). Since we are calling the SLM twice now, we need to log how this impacts CPU/MPS memory limits.
- **Zotero Management:** Add new literature references covering "Query Rewriting in RAG" and "Zero-shot Intent Extraction" to back up our methodology pivot in the final paper.

M4: Praisyl (Data Engine & Failsafe Verification)

- **Synthesizing Messy Data:** Update `src/generate_rlaif_data.py`. Instead of generating neat queries, simulate "messy farmer paragraphs" paired with their clean NLI triplets so we have realistic edge-case evaluation data.
- **Invariant Python Suite:** Solidify the `verify_aquaculture_invariants` script to ensure our biological ground truths (like pH 7.5-8.5 boundaries) cannot be broken by weirdly phrased user inputs.

Task Flow:

Step 1: Praisyl (The Starting Block)

- **When to start:** Immediately.
- **What they do:** Update the RLAIF generation script to output "messy farmer paragraphs" paired with their clean declarative hypotheses, and enforce the biological invariant filters.
- **The Hand-off Trigger:** Praisyl pushes the updated evaluation dataset to the repository. This instantly unblocks Khushi and Meghana.

Step 2: Khushi (Core Logic & Routing)

- **When to start:** As soon as Praisyl pushes the dataset.
- **What they do:** Write the strict zero-shot extraction prompt for Qwen. Then, update the `decision_router.py` script so the DeBERTa cross-encoder evaluates the *flattened hypothesis* against the chunks, locking in the 0.75 / 0.40 / 0.30 probability thresholds.
- **The Hand-off Trigger:** Khushi pushes the extraction prompt and working NLI router to the repository.

Step 3: Meghana (Dual-SLM Wiring & UI)

- **When to start:** In parallel with Khushi, but finalizes once Khushi's extraction prompt is ready.
- **What they do:** Wire `llama.cpp` to execute the extraction prompt *before* retrieval, and apply the GBNF grammar for the final generation pass. Update the Streamlit UI to handle the new **CLARIFY** conversation loop.
- **The Hand-off Trigger:** The end-to-end Streamlit app successfully runs the dual-SLM pipeline locally without crashing.

Step 4: Jyothish (Benchmarking & Documentation)

- **When to start:** Ramps up as Meghana's UI stabilizes.
- **What they do:** Profile the new hardware load to see how running two SLM calls impacts Time-to-First-Token (TTFT) and RAM usage. Draft the updated cascaded architecture diagrams and curate Zotero references for query rewriting.